

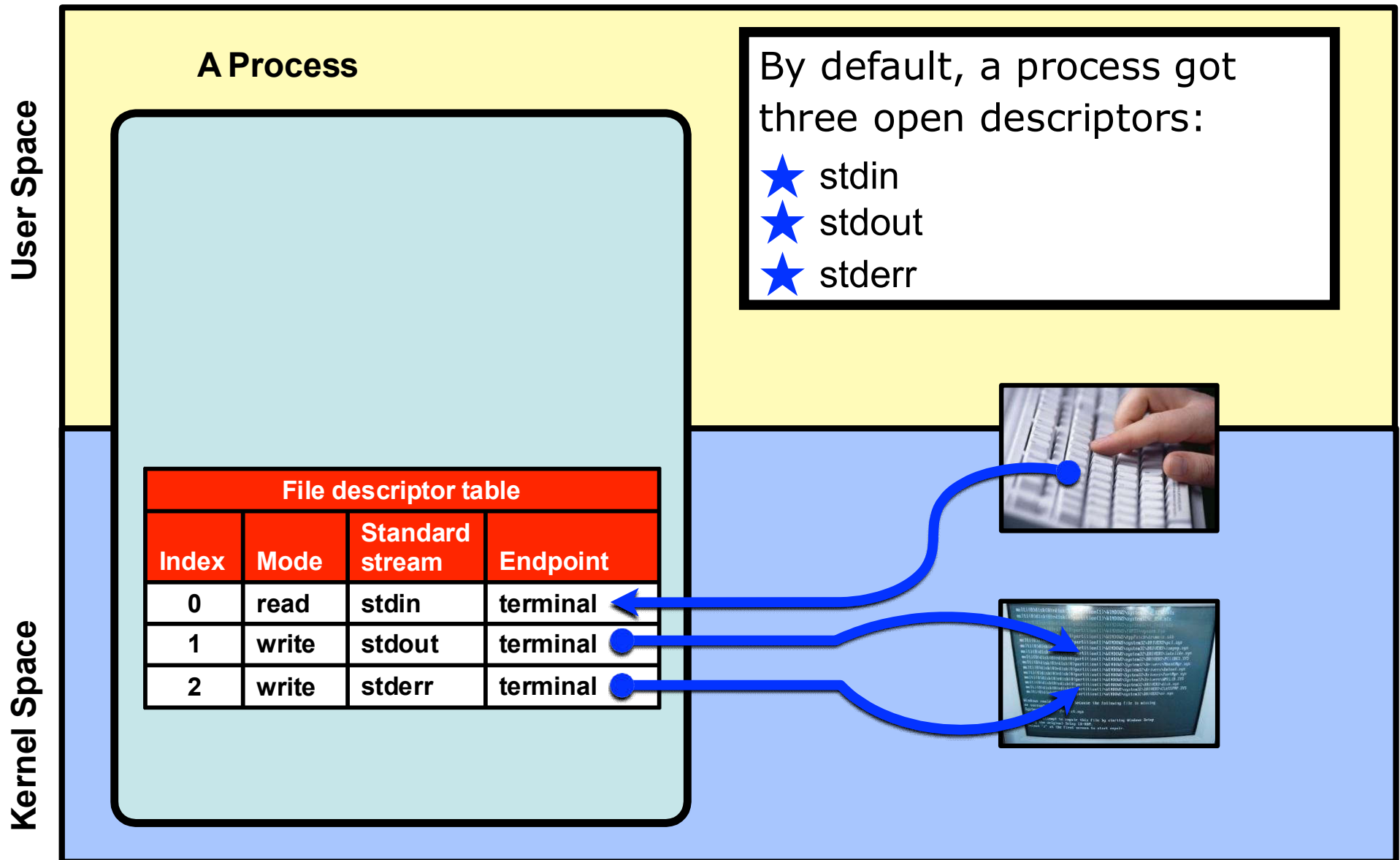
File Descriptors

File descriptor values for the standard I/O streams

File descriptor value	Standard stream	Mode	Endpoint
0	stdin	read	terminal
1	stdout	write	terminal
2	stderr	write	terminal



File Descriptor Table : The kernel keeps a table with information about a process's open file descriptors.



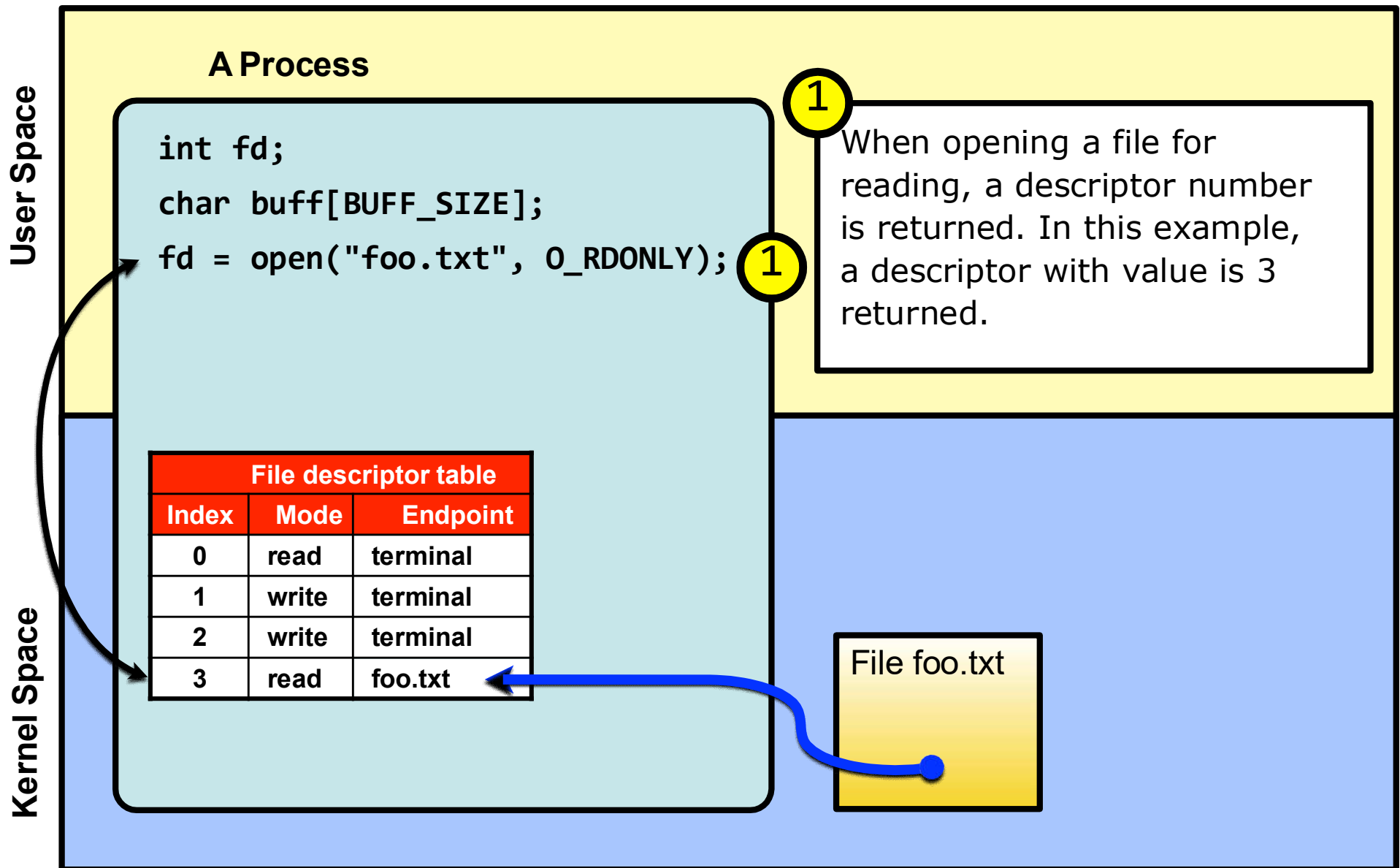
open()

Open is a **system call** that is used to **open** a new **file** and **obtain** its **file descriptor**.

```
int open(const char *path, int oflags);
```

int path	The relative or absolute path to the file that is to be opened.
int oflags	A bitwise 'or' separated list of values that determine the method in which the file is to be opened (whether it should be read only, read/write, whether it should be cleared when opened, etc).
return value	The file descriptor for the opened file. The file descriptor returned is always the smallest integer greater than zero that is still available. If a negative value is returned, then there was an error opening the file.

Open a file for reading using the `open()` system call



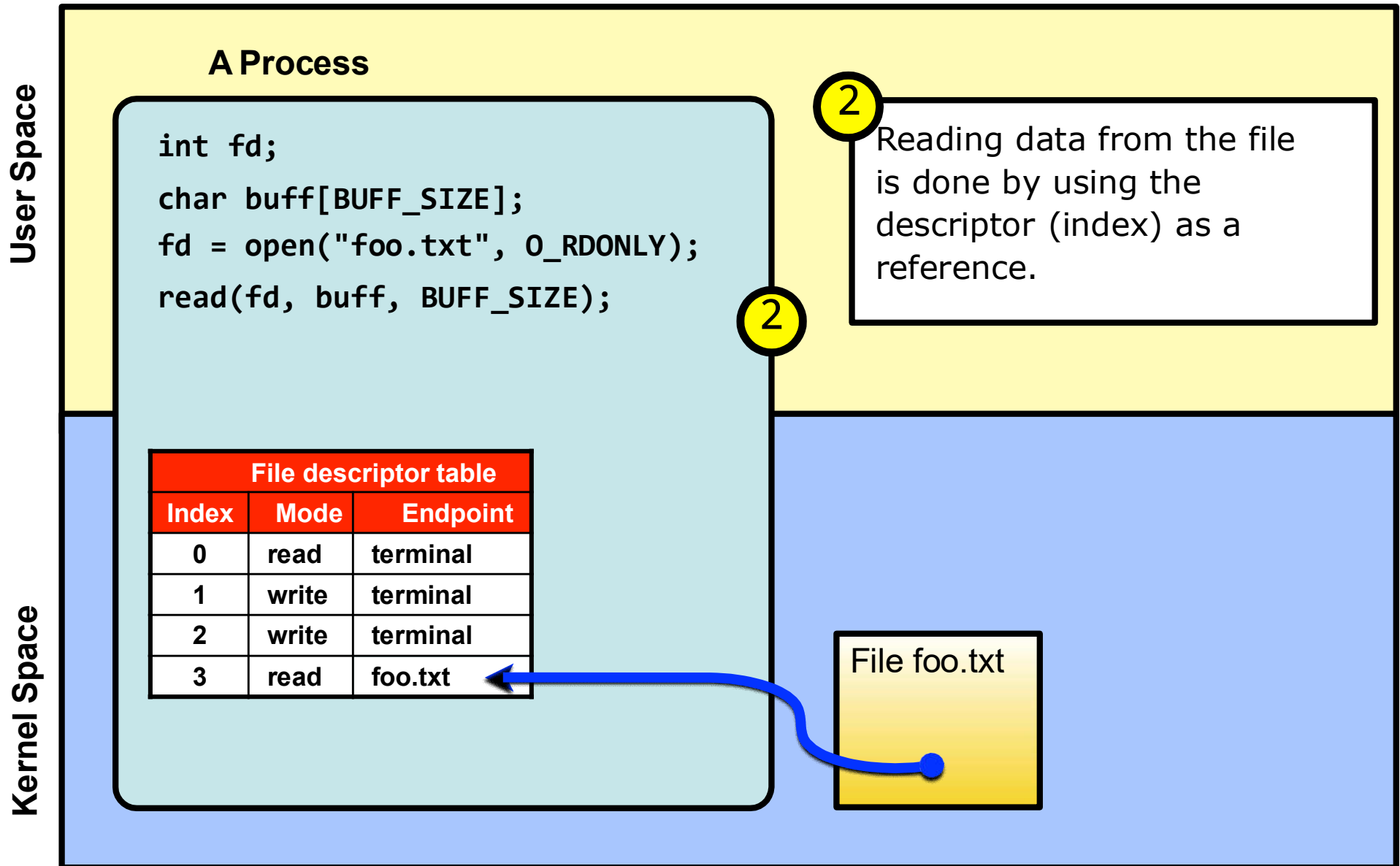
read()

Read is a system call used to read data from a file into a buffer.

```
size_t read(int fd, void *buf, size_t nbytes);
```

int fd	The file descriptor of where to read the input. You can either use a file descriptor obtained from the open system call, or you can use 0, 1, or 2, to refer to standard input, standard output, or standard error, respectively.
void * buf	A character array (buffer) where the read content will be stored.
int nbytes	The number of bytes to read before truncating the data. If the data to be read is smaller than nbytes, all data is saved in the buffer.
return value	On success, the number of bytes read is returned (zero indicates end of file). If value is negative , then the system call returned an error .

Reading data from a file using the **read()** system call



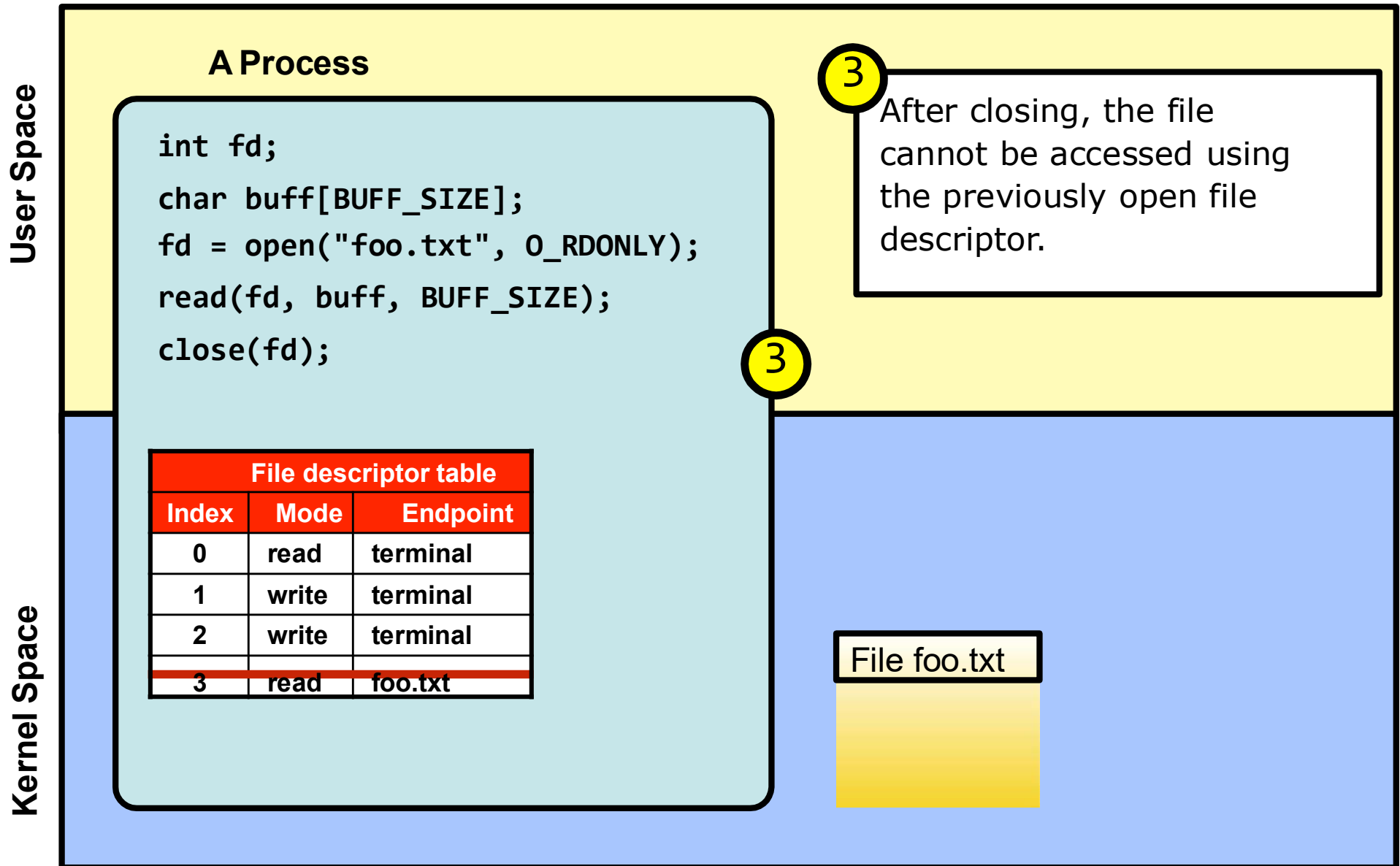
close()

Close is a system call that is used to close an open file descriptor.

```
int close(int files);
```

int files	The file descriptor to be closed.
return value	Returns a 0 upon success, and a -1 upon failure. It is important to check the return value, because some network errors are not returned until the file is closed.

Closing an open file using the **close()** system call



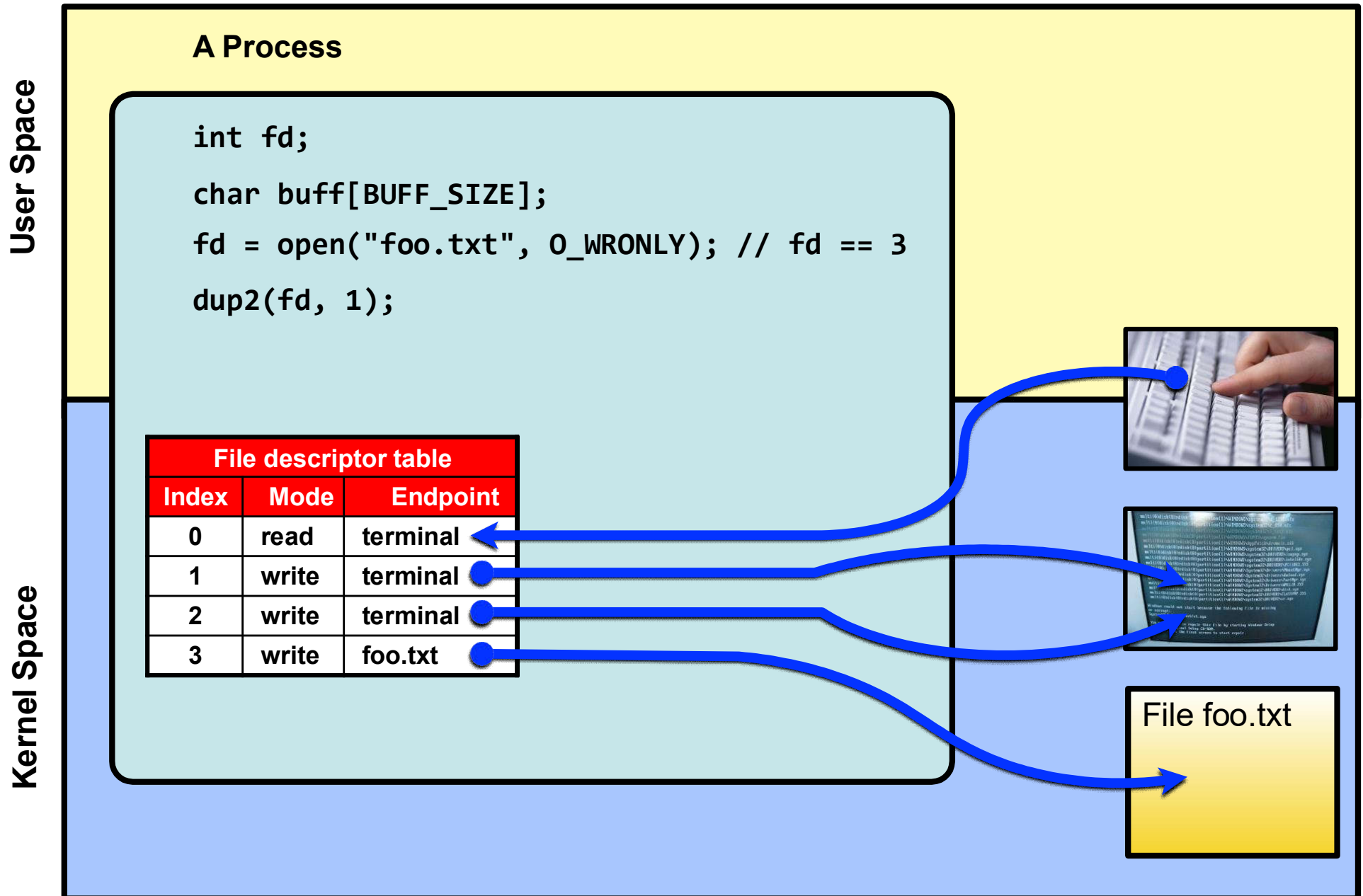
dup2() [used to redirect I/O]

Copies one file descriptor entry to another entry in the file descriptor table.

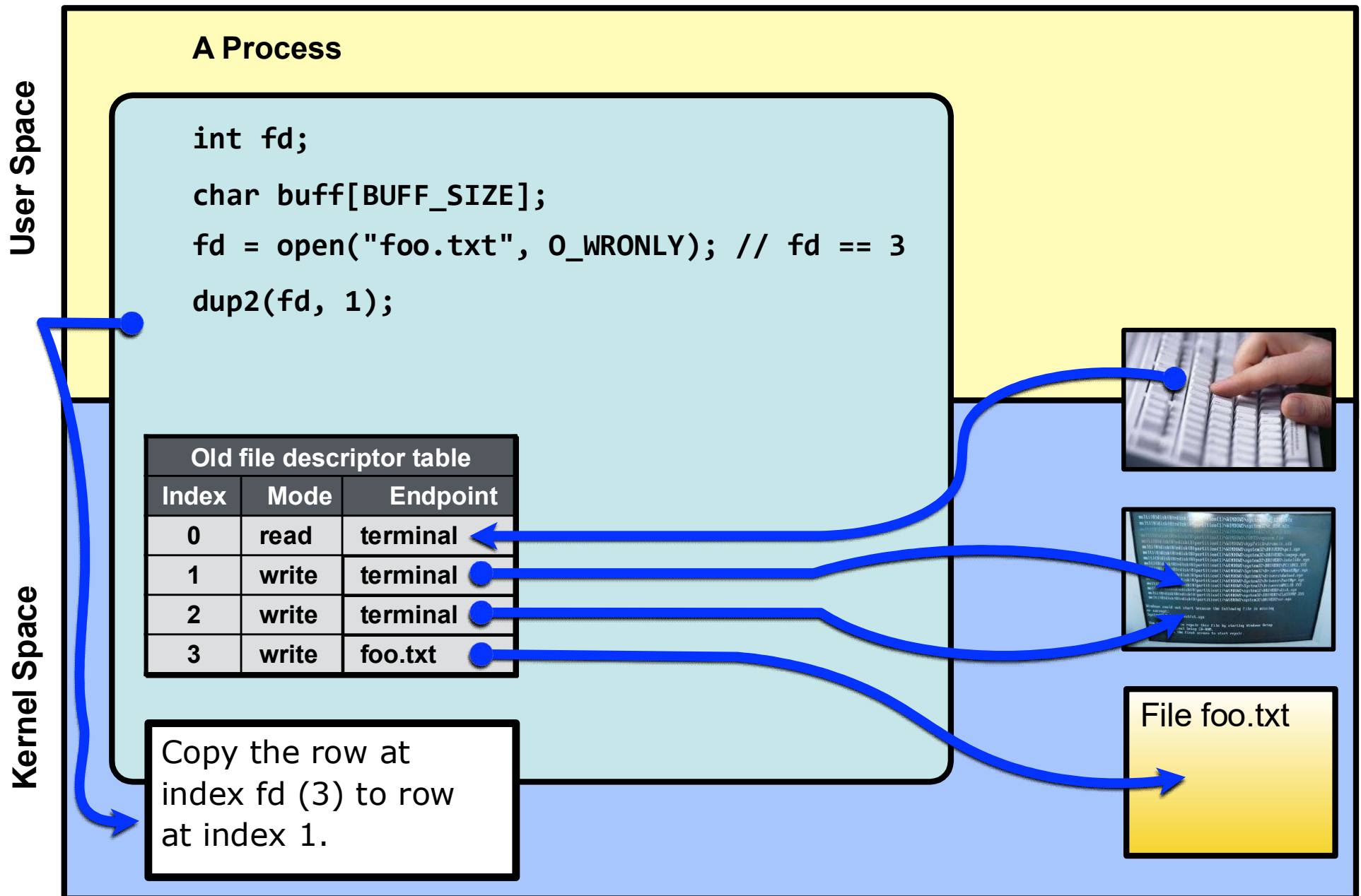
```
int dup2(int src, int dst);
```

int src	The source file descriptor. This remains open after the call to dup2.
int dst	The destination file descriptor. This file descriptor will point to the same file as src after this call returns. If the file descriptor dst was previously open, it is silently closed before being reused.
return value	dup2 returns the value of the second parameter (dst) upon success. A negative return value means that an error occurred

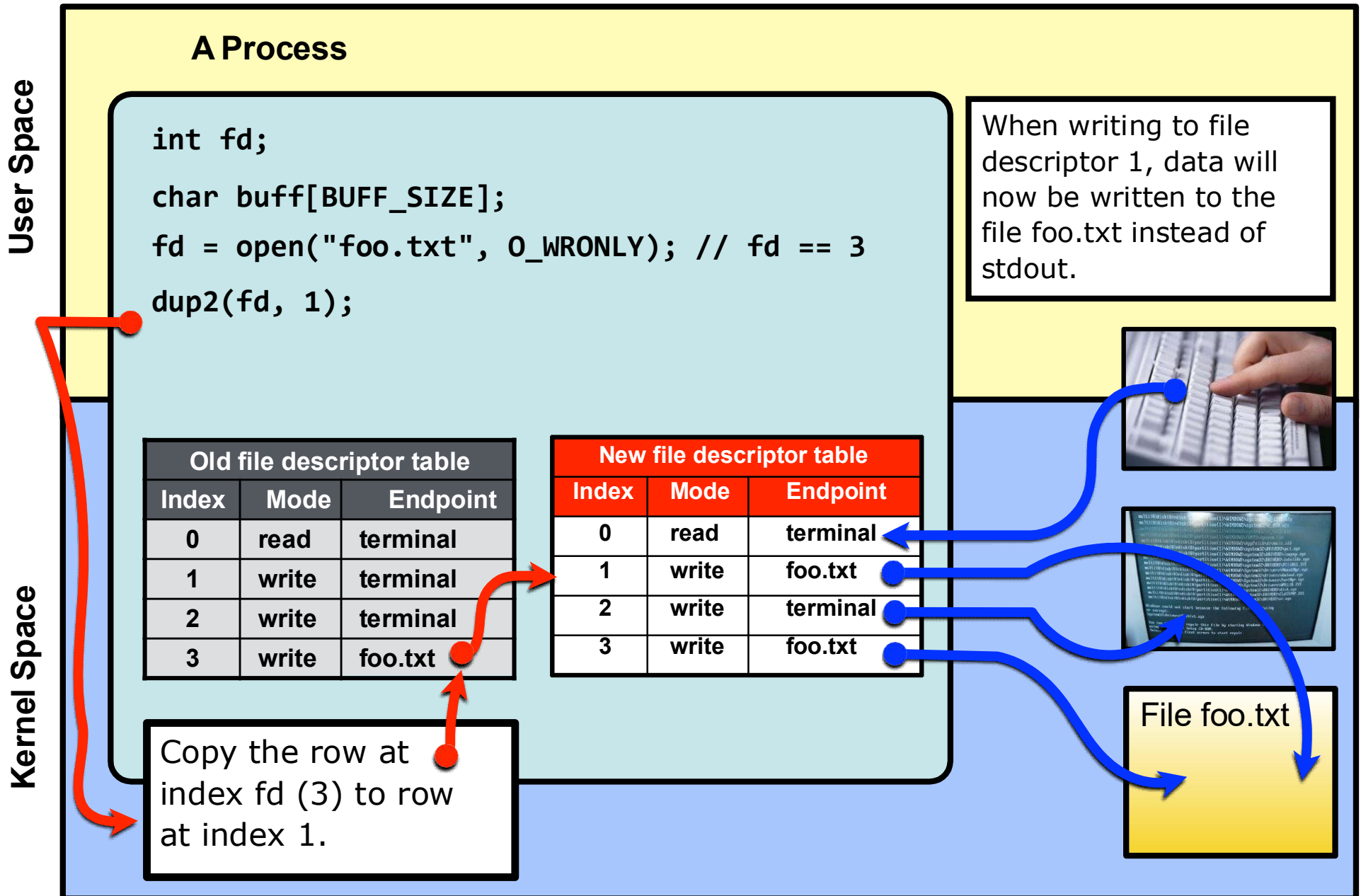
Redirect stdin using the **dup2()** system call



Redirect stdin using the **dup2()** system call



Redirect stdin using the **dup2()** system call



Pipes

- Pipes are used to redirect a stream from one program to another.
- When a program's standard output is sent to another through a pipe, the first program's data, which is received by the second program, will not be displayed on the terminal.
- Only the filtered data returned by the second program will be displayed.
- The Linux *pipe* is represented by a vertical bar `* | *`
- Examples:
 - `ls -l | wc -l`
- For more details, check main slides + multiple code exercises uploaded